

Egison Core

Demand-Directed Typing の形式仕様

概要

本仕様は、first-class matcher を生成する型と消費する型を分け、消費位置でだけ coercion を選ぶ Egison core を定式化する。中心課題は、停止する実行可能推論、唯一の source typing、推論状態を消去した動的安全性を、一つの設計として接続することである。各規則だけでなく、fresh supply, prevailing substitution, Origin ledger, terminal audit が必要になる理由と、soundness, acceptance completeness, state erasure, 安全性の依存関係を明示する。

目次

1	目的, 保証, 読み方	2
2	設計の必然性と依存関係	4
3	構文と判断	7
4	一回の checking cut	10
5	式の synthesis 規則	12
6	Pattern と matcher の規則	14
7	機械化された性質と接続	17

1 目的, 保証, 読み方

1.1 解く問題

Egison core では matcher が first-class value であり, 式は matcher を生成することも, matcher を必要とする位置へ渡されることもある. この二つを同じ型だけで表すと, 通常の型等価性と暗黙 coercion のどちらを選ぶかが, unification の失敗や特定の source 構文に依存してしまう. 本仕様は producer type $\text{Matcher}_{\kappa\tau}$ と consumer expectation $\text{MatcherSlot}_{\kappa\tau}$ を分け, resolved expected head が slot である checking cut だけを coercion demand とする.

この calculus が同時に満たすべき目標は次の四つである.

1. **source acceptance を一つにする.** program が型付くことは SourceTyping だけで定め, 動的メタ理論用の内部 invariant を第二の source type system にしない.
2. **推論を実行可能かつ fail closed にする.** raw traversal は停止し, 有限 validator を通った結果だけを公開する. caller に solver trace や bridge certificate を要求しない.
3. **coercion を demand-directed かつ no-guess にする.** 未解決型を coercion のために slot へ推測せず, 通常単一化の失敗後に別 branch を試さない.
4. **source typing を動的安全性へ接続する.** 推論順序に依存する状態を消去し, value typing, preservation, matching safety が消費できる state-free な TypingInvariant を構成する.

1.2 公開保証

公開推論器と source typing の二方向, および closed program の安全性は次の形で機械化されている.

$$\begin{aligned} \text{infer}(\widehat{\Sigma}, \Gamma, e) = \text{some } r &\implies \text{SourceTyping}(\widehat{\Sigma}, \Gamma, e, r.\text{resolvedTarget}), \\ \text{SourceTyping}(\widehat{\Sigma}, \Gamma, e, \tau) + \text{FrozenSigWF}(\widehat{\Sigma}) &\implies \text{infer}(\widehat{\Sigma}, \Gamma, e) \neq \text{none}, \\ \text{SourceTyping}(\widehat{\Sigma}, \emptyset, e, \tau) + \text{FrozenSigWF}(\widehat{\Sigma}) &\implies \text{SafeResult}_{\text{Source}}(\widehat{\Sigma}, e, \tau, \mathcal{S}_F). \end{aligned}$$

従って FrozenSigWF の下で, 一般 context の source typability と公開推論の成功は同値である. ただし SourceTyping 自体が terminal audit を定義に含む. 従ってこの同値と受理完全性は, audit 済みの SourceTyping に相対的であり, raw derivation, audit を除いた独立の宣言的型付け, または実行時に安全な全 program に対する完全性ではない. 同じ source の二つの SourceTyping target は, 全 residual capability / target metavariable の局所 renaming を法として一意である. さらに, source target に現れる二 sort metavariable だけを変更する有限 support substitution で instance preorder を定めると, inferType の返値は一般 context でも principal である. open term については, 実行 run の resolved context / target と, 各 derivation の terminal-normalized context / target を同じ paired substitution で比較する相対 principality も成立する.

pattern-free で capability-inert な Damas–Milner 断片についても二方向の境界を機械化する. closed source fragment の typing は DM typing へ消去でき, 逆に任意の DM typing derivation は, その context を埋め込んだ公開推論器で受理される:

$$\text{FrozenSigWF}(\widehat{\Sigma}) \wedge \text{DM.Typing}(\Gamma, e, \tau) \implies \text{inferenceSucceeds}(\widehat{\Sigma}, \text{emb}(\Gamma), e) = \text{true}.$$

この保証は acceptance であり, DM derivation が選ぶ τ と公開推論器の返値型の構文的一致ではない. 後者は principality / instance preorder による別の比較問題である.

1.3 判断の役割を分ける

構成要素	役割
SourceTyping	terminal audit を含み, source acceptance を定義する唯一の公開 judgment
ExprDeriv	成功した実行 trace を規則木へ戻す proof-relevant な内部 reconstruction
TypingInvariant	supply, substitution, ledger を消去した動的メタ理論用の内部 invariant
ValueTy と matching judgments	evaluation と matching machine の preservation / progress を述べる runtime 側の判断

依存方向は

$$\text{SourceTyping} \xRightarrow{\text{state erasure}} \text{TypingInvariant} \xRightarrow{\text{dynamic metatheory}} \text{runtime safety}$$

である. TypingInvariant の存在から SourceTyping や推論成功を逆向きに推論しない.

1.4 非目標と現在の境界

一般の program termination は主張しない. recursion は singleton direct-self の単相 fix に限る. matcher literal は shape, catch-all order, binder 線形性, data-arm exhaustiveness, coverage をすべて要求する. 未解決 lambda domain を複数箇所でも共有したときの source-order 依存は正負回帰で固定する. これは no-guess と chronological state threading の意図された言語境界であり, source 要素の順序不変性は主張しない. また DM acceptance は推論成功だけを主張し, DM target と推論返値の構文的一致は非目標である.

terminal audit には, raw では受理されるが公開推論では拒否されるという, 意図した言語境界とは区別すべき既知の受理損失がある. 整形形式な signature の下で, 一要素 tuple の要素を引数 matcher へ委譲する閉じた lambda は raw 推論に成功するが, 引数 slot から借りた capability variable が局所 finalization 後に観察不能な K へ特殊化される. 9 個の terminal check は

(true, true, true, true, true, true, true, false, true)

であり, matcher-finalization check だけが失敗して公開推論は拒否する. compile-time guard がこの具体計算を検査し, 通常の定理が任意の raw 成功等式から demand-directed derivation を構成する. 閉 lambda が closure へ評価されることも証明済みであるが, audit と独立な宣言的型付けに基づく安全性は未証明である. 従ってこの例を一般的な意味で安全とする定理や, terminal audit による受理損失の正確な範囲は公開保証に含めない.

修正候補は, matcher 自身が観察した構造と, 引数 slot から完全な値として借りた leaf の由来を finalization まで保持することである. 借りた variable も一律に freeze する方法は意図した後続特殊化を失い, すべての opaque leaf を一律に許す方法は matcher 自身が内部構造を観察する場合まで受理し得るため, どちらも採用しない.

1.5 読み方

第 2 章は, 二 sort 型, 三つの推論状態, Origin certificate, terminal audit, validator が必要になる理由と, 公開定理までの依存関係を示す. 第 3 章で構文と判断を定義し, 第 4 章で一回の checking cut を定式化する. 第 5 章は通常の式, 第 6 章は pattern と matcher literal の規則である. 第 7 章は局所代数を traversal へ持ち上げ, soundness, acceptance completeness, state erasure, 安全性, 受理同値, target 一意性, principality へ接続する. Lean module と回帰の索引は ../docs/details.md に譲る.

設計だけを追う読者は第 2 章, 第 4 章, 第 7 章を読めばよい. 規則を実装と照合する場合は第 3-6 章, 証明の依存を確認する場合は第 2.4 節と第 7 章を読む.

2 設計の必然性と依存関係

本章は規則を導入する前に、どの問題に対してどの構成要素が必要かを分離する。後続章はここで示す依存方向に従う。

2.1 Matcher と MatcherSlot を分ける理由

matcher producer と matcher consumer を同じ型だけで表すと、coercion を選ぶ根拠が型に現れない。その場合、特定の構文を coercion site として列挙するか、通常単一化が失敗した後で coercion を試す必要がある。前者は first-class な関数引数に一般化できず、後者は rollback, solver の探索順, fuel に受理結果を依存させる。

そこで producer を $\text{Matcher}_{\kappa\tau}$, consumer expectation を $\text{MatcherSlot}_{\kappa\tau}$ とし, checking cut で resolved expected head を観測する。これにより

設計上の問題	採用する表現	得られる性質
通常等価性と coercion の区別	expected head の MatcherSlot	branch 選択が unification 失敗に依存しない
coercion site の一般化	構文ではなく checking judgment	<code>matchAll</code> と通常の関数適用が同じ規則を使う
producer の保護	one-way matcher-to-slot solve	consumer demand が producer capability を書き換えない

非恒等 coercion の終点は常に slot である。matcher-headed expected type では ordinary equality だけを許す。expected head が未解決なら構造を推測せず, ordinary branch を選ぶ。

2.2 三つの推論状態を分ける理由

demand-directed judgment は $q; S; \Omega$ を左から右へ thread する。三成分は互いに代用できない。

状態	記録するもの	独立に必要な理由
q	二 sort の次の fresh 番号	source 判断と実行走査の割当順序を一致させ、変数の boundedness と fuel recursion を証明する
S	その cut までの constraint 解	子を左から右へ接続し、expected head, context, 一般化を同じ chronological state で解釈する
Ω	capability variable の生成由来と solve 権限	同じ構文形の variable でも、rigid, rename-only, structural-flexible を区別して producer strengthening を拒否する

S だけでは capability variable が scheme producer 由来か局所 consumer 由来かを復元できない。型の shape だけから Ω を再計算することもできないため、ledger は substitution とは別に保持する。一方、 Ω は coercion demand を決める。demand は expected slot head, solve 権限は origin という別軸である。

2.3 二種類の証明境界

実行可能推論と関係的 source typing は同じ policy を異なる形で保持する。

構成要素	必要になる理由	後続で可能になること
raw demand-directed derivation	規則木と三状態の入出力を記録する	traversal の構造帰納, state factorization
intrinsic Origin certificate	各 solve と freeze が ledger policy に従うことを関係的に証明する	source 側の no-guess / producer protection
terminal audit	局所 cut では後続 solve に安定しない事実を root 終端で固定する	state erasure と validator completeness
executable trace	solver, allocation, event emission を計算として記録する	reconstruction と有限 validator
terminal validator	raw 成功のうち全 bridge 条件を満たす run だけを公開する	成功等式だけを premise とする soundness
TypingInvariant	推論履歴を runtime typing から切り離す	preservation と matching safety

Origin certificate は実行 trace のコピーではなく、SourceTyping derivation 自身が solve の admissibility を持つために必要である。validator は逆に、実行 run がその関係的条件を満たすことを有限に検査する。この二つを混同しないため、soundness は trace から Origin を再構成し、completeness は Origin と terminal audit から成功 trace を再構成する。従ってここでいう completeness は、terminal audit を定義に含む SourceTyping に相対的な主張である。raw derivation が常に audit を満たすことや、audit と独立に定めた宣言的型付けに対する完全性は含まない。

2.4 terminal audit を置く三箇所

局所 cut で成立しても、外側の後続 substitution を適用すると再確認が必要な事実がある。すべての node に任意の future suffix への安定性を要求すると、let の generalized set が変わり得るため強すぎる。そこで公開 root の終端 substitution に固定した audit を、raw derivation と同じ木構造で持つ。

■let の局所 cut だけを考えた例。audit のない let cut が何を許すかを考える。空の signature / context, cut substitution id, value type $\tau_1 = \alpha_0 \rightarrow \alpha_1$ に対し、cut での generalization は $\sigma_{\text{cut}} = \text{Gen}(\emptyset, \emptyset, \tau_1) = \forall a b. a \rightarrow b$ である。body の後続 solve が生成したかを問わず、任意の後続 suffix $R = [\alpha_1 \mapsto \alpha_0]$ を許すと、binder は既に close されているので $R\sigma_{\text{cut}} = \sigma_{\text{cut}}$ のままである。しかし root で再計算すべき scheme は $\text{Gen}(\emptyset, \emptyset, R\tau_1) = \text{Gen}(\emptyset, \emptyset, \alpha_0 \rightarrow \alpha_0) = \forall a. a \rightarrow a$ となる。従って $R\sigma_{\text{cut}} \neq \text{Gen}(\emptyset, \emptyset, R\tau_1)$ であり、audit なしでは domain と codomain を独立に instantiate できる過剰な scheme を残してしまう。let の terminal audit はこの不一致を拒否する。ただし、これは任意の後続 suffix に対して局所 cut だけを考えた代数的な例であり、この suffix を実際の source traversal が生成することを示す source program ではない。matcher と pattern constructor では、同じ再検査原理を terminal hole evidence と capability compatibility へ適用する。

node	局所 cut だけでは足りない事実	root 終端で再確認するもの
let	body の後続 solve で context と value type が変わる	終端 context / value type から再計算した generalization
matcher literal	hole capability と final capability が rename され得る	evidence, shape, clause capability, exhaustiveness, coverage
pattern constructor	argument dual と result capability が後続 solve を受ける	終端 substitution 後の CapCompatible

■matcher finalization で実際に到達する terminal audit による受理損失。 List 用 signature へ、観察不能 (opaque) な capability constructor K を引数にだけ使う整形式な data constructor

$$\text{consumeK} : \text{Matcher K Int} \rightarrow \text{Witness}$$

を加える。一要素 tuple を分解し、その要素を引数 matcher へ委譲する二節 matcher を *fix self argument* で包むと、次の閉じた lambda が得られる。

$$\lambda m. (\text{consumeK } m, (\text{fix self argument. matcher}[\text{tuple}_1 \mapsto \text{argument}; \text{catch}]) m).$$

self は使わず、*fix* は引数 slot の局所 capability variable を作るためだけに用いる。整形式 signature の下で *inferRaw* は成功し、raw target は

$$\text{Matcher K Int} \rightarrow (\text{Witness} \times \text{Matcher prod(K) prod(Int)})$$

である。9 個の terminal check は順に

$$(\text{true}, \text{true}, \text{true}, \text{true}, \text{true}, \text{true}, \text{true}, \text{false}, \text{true})$$

となり、8 番目の *traceFinalizationSuffixCheck* だけが失敗して公開 infer は拒否する。matcher finalization は、引数 slot から借りた capability variable を freeze 対象から除く。局所検査後の application がこの variable を K へ特殊化すると、終端検査は委譲先の K まで再帰的に観察しようとして拒否する。しかし matcher 自身が観察するのは外側の一要素 tuple だけであり、要素の観察は引数 matcher へ委譲される。これは raw では受理されるが公開推論では拒否されるという、terminal audit による具体的な受理損失である。

形式化では、compile-time guard が raw 成功、公開拒否、型、9 検査の結果を計算し、通常の定理が任意の raw 成功等式から対応する demand-directed derivation を構成する。さらに閉 lambda が closure へ評価されることを証明している。audit と独立な宣言的型付けと、それに基づく安全性定理はまだないため、この program を一般的な意味で安全とする定理や、audit による損失の正確な範囲は未確立である。修正時には、matcher 自身が観察した構造と、引数 slot から完全な値として借りた capability の由来を finalization まで保つ必要がある。借りた variable も一律に freeze すると意図した後続特殊化を失い、すべての opaque capability を許すと matcher 自身が opaque な内部構造を観察する場合まで受理し得る。

variable node は追加 audit を持たない。canonical scheme の finite opening と substitution transport から終端 lookup との一致を代数的に導く。

2.5 公開定理までの依存グラフ

証明は局所代数から公開定理へ次の順で積み上げる。

$$\begin{array}{c} \text{exact solver} + \text{ledger admissibility} + \text{canonical scheme laws} \\ \Downarrow \\ \text{全 raw family の state extension} + \text{chronological factorization} + \text{solved-form 保存} \\ \Downarrow \\ \text{Origin tree} + \text{terminal audit} + \text{executable event coverage} \\ \Downarrow \\ \text{infer} \Rightarrow \text{SourceTyping} \quad \text{SourceTyping} \Rightarrow \text{infer} \quad \text{SourceTyping} \Rightarrow \text{TypingInvariant} \\ \Downarrow \\ \text{受理同値} \cdot \text{決定可能性} \cdot \text{target 一意性} \quad \text{SourceTyping による公開安全性} \end{array}$$

FrozenSigWF は source-facing な唯一の global signature 条件である。受理完全性には terminal facts を実行 state へ輸送する closedness と canonical arm checker を与え、安全性には scheme closedness と動的整合性を与える。低レベル補題は必要な field を明示してよいが、公開 caller へ別 premise として露出しない。

3 構文と判断

本章は、第 2 章で理由を与えた構成要素を形式的な記号へ落とす。型と canonical scheme を定めた後、source 構文、signature / context、三状態、判断 family、instantiation / generalization を導入する。ここで定義する記号が第 4-6 章の規則の入力となり、canonical scheme laws と supply bounds が第 7 章の reconstruction、state erasure、completeness に使われる。

3.1 型とスキーム

capability κ と target type τ は別 sort である。matcher と slot は同じ二つの index を持つが、前者は producer、後者は consumer expectation を表す。

$$\begin{array}{ll}
\kappa ::= \text{Any} \mid \chi \mid \hat{s} \mid K \bar{\kappa} \mid (\kappa_1, \dots, \kappa_n) & \kappa : \text{Cap}, \\
\tau ::= \alpha \mid \hat{a} \mid \mathbf{1} \mid \text{Int} \mid \text{Bool} \mid K \bar{\tau} \mid (\tau_1, \dots, \tau_n) \mid \tau_1 \rightarrow \tau_2 & \tau : \text{Type}, \\
& \mid \text{Matcher } \kappa \tau \mid \text{MatcherSlot } \kappa \tau \\
\sigma ::= \forall(\bar{\chi} : \text{Cap}).\forall(\bar{\alpha} : \text{Type}).\tau & \text{expression scheme,} \\
\eta ::= \kappa \triangleright \tau & \text{pattern dual,} \\
\varsigma ::= \forall(\bar{\chi} : \text{Cap}).\forall(\bar{\alpha} : \text{Type}).(\bar{\eta} \Rightarrow \eta) & \text{pattern-function scheme.}
\end{array}$$

χ, α は flexible metavariable, $\hat{s}, \hat{a}$ は rigid skolem である。Any は one-way capability matching の consumer 側でだけ wildcard として働き、対称 unification では通常の rigid constructor である。mono τ は量化 binder を持たない scheme を表す。

上の $\forall\bar{\chi}.\forall\bar{\alpha}$ は紙面上の表記である。Lean の canonical Scheme は binder 数と PolyCap/PolyTy payload を持ち、bound occurrence を Fin index、自由な solver metavariable を CapVar/TyVar として別 datatype で表す。通常型 κ, τ と unifier には bound constructor を追加しない。従って metavariable substitution は scheme の自由 occurrence だけを書き換え、bound index を捕捉できない。generalization は選んだ metavariable を直ちに finite index へ close し、instance は有限 opening を通してのみ通常型へ戻す。payload は構成時から alpha-normal form である。

3.2 ソース構文

$$\begin{array}{ll}
e ::= x \mid \lambda x.e \mid e_1 e_2 \mid \text{let } x = e_1 \text{ in } e_2 \mid \text{fix } f x.e \mid n & \\
& \mid (e_1, \dots, e_n) \mid C \bar{e} \mid \text{op}(\bar{e}) \mid \text{something} \\
& \mid \text{matcher } cls \\
& \mid \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e, \\
p ::= \$x \mid _ \mid \#e \mid \tilde{x} \mid c \bar{p} \mid p_1 \& p_2 \mid p_1 \mid p_2 \mid f \bar{p} \mid (\bar{p}), & \\
pp ::= \$ \mid _ \mid \#\$x \mid c \bar{pp} \mid (\bar{pp}), & \\
dp ::= \$x \mid _ \mid C \bar{dp} \mid (\bar{dp}), & \\
cl ::= pp \text{ as } M \text{ with } [dp_j \rightarrow N_j]_{j=1}^r, & \\
cls ::= [cl_i]_{i=1}^m. &
\end{array}$$

c は pattern constructor, C は data constructor である。surface match は matchAll と head への elaboration であり、core primitive ではない。pattern-function declaration は source expression の typing より前に freeze (後の推論で構造を強められないよう固定) され、canonical signature の一部になる。

3.3 シグネチャ・文脈・状態

$\hat{\Sigma} = (\hat{\Sigma}_D, \hat{\Sigma}_P, \hat{\Sigma}_F, \hat{\Sigma}_{prim}, Obs_{\hat{\Sigma}})$ $\Gamma ::= \emptyset \mid \Gamma, x : \sigma$ $\Phi ::= \emptyset \mid \Phi, x : \eta$ $\Delta ::= \emptyset \mid \Delta, x : \tau$ $q = (q_\kappa, q_\tau)$ $S = (C, T)$ $\Omega : \chi \rightarrow \{\text{rigid}, \text{renameOnly}, \text{structuralFlexible}\}$	freeze 済み canonical signature, expression scheme context, pattern-parameter context, monomorphic pattern bindings, next capability / target variable numbers, prevailing paired substitution, capability-origin ledger.
--	---

q_κ と q_τ は独立な自然数 counter であり、それぞれ次に未使用の capability metavariable と target metavariable の番号を保持する。以下では

$$\chi_{q_\kappa} : \text{Cap}, \quad \alpha_{q_\tau} : \text{Type}, \quad q + \langle i, j \rangle = (q_\kappa + i, q_\tau + j)$$

と書く。従って capability variable を一つ生成すれば supply は $q + \langle 1, 0 \rangle$, target variable なら $q + \langle 0, 1 \rangle$, 両方を一つずつ生成する pattern variable 規則では $q + \langle 1, 1 \rangle$ となる。番号空間は sort ごとに別なので、 χ_3 と α_3 は別の変数である。closed wrapper の初期 supply は、signature と context に既出の番号より各 counter が大きくなるように選ぶ。 S の作用は capability を先に、target substitution を後に適用する。

$$S\kappa = C\kappa, \quad S\tau = T(C\tau), \quad S\Gamma = \{x : S\sigma \mid x : \sigma \in \Gamma\}.$$

substitution delta は seq で時系列順に prevailing substitution へ合成する。後段の capability action は前段の target range 内部にも作用するため、二 component を独立に合成しない。

Ω は capability variable がどこで生成され、現在どの solve を許すかを記録する。未登録の key は rigid である。scheme / dual instance の binder image は renameOnly, constructor / primitive instance と fresh consumer は structuralFlexible になる。export は外へ残る像の structural leaf だけを renameOnly へ移す。matcher finalization は最終 capability に現れる matcher-owned な explicit ledger key を調べる。ただし active context の top-level slot demand または direct-self の引数 slot から借りた leaf は新規 freeze 対象から除き、self の結果だけに現れる leaf は除外しない。

■三状態が型推論を分ける例。三状態は solver metavariable を分類し、capability 構文の rigid skolem $\hat{s}$ とは区別される。役割は次の型推論例に現れ、それぞれの受理・拒否境界は機械化されている。

局所特殊化。 $\text{Pack} : \forall \chi. \forall \alpha. \text{Matcher } \chi \alpha \rightarrow \text{Packed}$ に $\text{something} : \text{Matcher Any } \beta$ を渡すと、constructor instance の χ_f は structural-flexible なので $\chi_f \mapsto \text{Any}$ と解け、Packed を推論する。rigid または rename-only では、generic declaration の正当な特殊化を誤って拒否する。

export 後の rename。 引数なし constructor を $\text{makeF} : \forall \chi. \text{Matcher } \chi \text{Int} \rightarrow \text{Int}$, $\text{makeM} : \forall \chi. \text{Matcher } \chi \text{Int}$ とする。独立に生成された χ_1, χ_2 は result に残るため export で rename-only になる。 $\text{makeF}()$ $\text{makeM}()$ は $\chi_2 \mapsto \chi_1$ だけを必要とし、Int を推論する。完全な rigid では fresh 名の違いだけで誤って拒否する。

producer strengthening の拒否。 consumer が $\text{Matcher List}(\text{Any}) \text{Int}$ を要求し、context に $\text{producer} : \forall \chi. \text{Matcher } \chi \text{Int}$ があるとする。lookup が作る χ_n は rename-only なので、 $\chi_n \mapsto \text{List}(\text{Any})$ を必要とする $\text{consumeProducer producer}$ は拒否される。producer が最初から $\text{Matcher List}(\text{Any}) \text{Int}$ なら成功する。value-flow image を structural-flexible にすると、利用箇所から producer capability を捏造してしまう。

外部仮定の固定。 入力 context の $r : \text{Matcher } \chi_r \text{Int}$ に自由な χ_r が現れる場合、 Matcher Any Int が必要な位置で r を検査しても $\chi_r \mapsto \text{Any}$ としてはならない。未登録の χ_r は rigid なので、caller の仮定を変えるこの constraint を拒否する。

以上より、structural-flexible は局所特殊化、rename-only は export 済み producer の alpha-renaming, rigid は外部仮定の保存を担う。rename-only な χ_n と structural-flexible な χ_f の等式は $\chi_f \mapsto \chi_n$ へ向ける。逆向きでは後続の structural solve が χ_n を間接的に強化できる。export と matcher finalization は外へ出る matcher-owned な structural leaf を rename-only へ一方向に freeze する。引数 slot から借りた leaf は call site で特殊化される consumer interface なので、matcher-owned な出力 leaf とは区別する。ledger は solver の解を制限するが、coercion branch の選択には使わない。

3.4 判断の一覧

$q; S; \Omega; \Gamma \vdash e \Rightarrow \tau \dashv q'; S'; \Omega'$	expression synthesis
$q; S; \Omega; \Gamma \vdash e \Leftarrow \rho \dashv q'; S'; \Omega'$	expression checking
$q; S; \Omega; \Gamma \vdash \bar{e} \Rightarrow \bar{\tau} \dashv q'; S'; \Omega'$	expression-list traversal
$q; S; \Omega; \Gamma; \Phi; \Delta \vdash p \Rightarrow \eta; \Delta' \dashv q'; S'; \Omega'$	user-pattern synthesis
$q; S; \Omega \vdash pp \Leftarrow \tau \Rightarrow \bar{\eta}; \Delta \dashv q'; S'; \Omega'$	primitive-pattern checking
$q; S; \Omega \vdash dp \Leftarrow \tau \Rightarrow \Delta \dashv q'; S'; \Omega'$	data-pattern checking
$q; S; \Omega; \Gamma \vdash cl \Leftarrow \tau \Rightarrow \bar{\eta} \dashv q'; S'; \Omega'$	matcher-clause inference
$q; S; \Omega; \dots \vdash \bar{a} \text{ or } \bar{cl} \dashv q'; S'; \Omega'$	arm / clause-list traversal

list judgment は空列で q, S, Ω をそのまま返す. cons では head の出力を tail の入力にし, binding context も左から右へ延長する. lookup または規則中の部分関数が失敗した場合, その規則は適用できない.

3.5 具体化と一般化

Inst_q と InstDual_q は supply-indexed に量化 binder を fresh variable へ置き換え, 次 supply も返す. expression context と pattern-function signature の量化 capability binder は capability variable へだけ instantiate され, その像を Ω に `renameOnly` として登録する. 一方, data / pattern constructor と primitive の scheme binder は `structuralFlexible` として登録し, 局所的な structural instance を許す. 利用側の demand だけを根拠に producer capability を constructor や product へ変えることはできない.

Gen は signature と context に自由な variable を除いて量化する.

$$\text{Gen}(\widehat{\Sigma}, \Gamma, \tau) = \forall(\text{fcv}(\tau) \setminus (\text{fcv}(\widehat{\Sigma}) \cup \text{fcv}(\Gamma))). \forall(\text{ftv}(\tau) \setminus (\text{ftv}(\widehat{\Sigma}) \cup \text{ftv}(\Gamma))). \tau.$$

scheme binder は局所的であり, 後段の substitution が導入した同番号の自由 variable を捕捉しない. let は value の synthesis 直後の $S\Gamma, S\tau$ を一般化する. さらに body の終端 substitution S' に対して

$$S'\sigma = \text{Gen}(\widehat{\Sigma}, S'\Gamma, S'\tau)$$

が成り立つことを Origin certificate に要求する. これは generalization 後の binder を後続 solve が遡及的に構造化しないための terminal stability 条件である.

4 一回の checking cut

本章の入力は, 第 3 章の二 sort 型, prevailing substitution S , Origin ledger Ω である. 出力は, 一つの checking cut で ordinary equality または slot-directed coercion のどちらを選び, どの chronological delta を後続規則へ渡すかである. この局所決定性が, 式と pattern で同じ checking を再利用し, 第 7 章で slot-demand theorem と solver correspondence を証明する基礎になる.

4.1 二つの alignment 判断

coercion を伴わない equality alignment と checking cut 全体を, それぞれ

$$S; \Omega \vdash \tau \doteq \rho \dashv S', \quad S; \Omega \vdash \tau \preccurlyeq \rho \dashv S'$$

と書く. どちらも constraint を解く局所 delta を入力 S の後へ時系列順に合成する. ledger Ω 自体は solve では変化しないが, delta が Ω に対して admissible であることを要求する.

$\text{ExactCapMGU}_\Omega$ と $\text{ExactPairedMGU}_\Omega$ は, capability と paired type constraint の proof-carrying MGU に origin admissibility を加えたものである. MGU の soundness と因子化に加え, constraint 外で恒等, range が constraint

内, solved form が冪等であることを要求する. さらに rigid key は固定し, renameOnly key は非 structural variable へしか写さない. OneWayDelta_Ω は producer を固定したまま consumer capability と target を解く exact delta であり, 同じ admissibility 条件を満たす.

4.2 通常の等価整合

matcher 同士または slot 同士では capability を先に解き, その delta を target へ作用させてから target constraint を解く. それ以外の型は paired type 全体を一度に解く.

$$\frac{S\tau = \text{Matcher } \kappa_1 \tau_1 \quad S\rho = \text{Matcher } \kappa_2 \tau_2 \quad \text{ExactCapMGU}_\Omega(\kappa_1, \kappa_2, C) \quad \text{ExactPairedMGU}_\Omega(C\tau_1, C\tau_2, T)}{S; \Omega \vdash \tau \doteq \rho \dashv \text{seq}(T, \text{seq}((C, \text{id}), S))} \quad [\text{ALIGN-MATCHER}]$$

$$\frac{S\tau = \text{MatcherSlot } \kappa_1 \tau_1 \quad S\rho = \text{MatcherSlot } \kappa_2 \tau_2 \quad \text{ExactCapMGU}_\Omega(\kappa_1, \kappa_2, C) \quad \text{ExactPairedMGU}_\Omega(C\tau_1, C\tau_2, T)}{S; \Omega \vdash \tau \doteq \rho \dashv \text{seq}(T, \text{seq}((C, \text{id}), S))} \quad [\text{ALIGN-SLOT}]$$

$$\frac{\text{alignPairClass}(S\tau, S\rho) = \text{ordinary} \quad \text{ExactPairedMGU}_\Omega(S\tau, S\rho, D)}{S; \Omega \vdash \tau \doteq \rho \dashv \text{seq}(D, S)} \quad [\text{ALIGN-ORDINARY-TYPES}]$$

4.3 Demand classifier と coercion の選択

demandClass($S\tau, S\rho$) は resolved type だけから productMatcherLift, slotTupleLift, matcherToSlot, slotToSlot, ordinary のいずれかを定める. product of matchers, product of slots の順に調べるため, 空 product は前者が優先される.

$$\frac{\text{productMatcherDuals?}(S\tau) = \text{some } \bar{\eta} \quad S\rho = \text{MatcherSlot } \kappa v \quad \text{OneWayDelta}_\Omega((\bar{\eta}.\bar{\kappa}), (\bar{\eta}.\bar{\tau}), \kappa, v, D)}{S; \Omega \vdash \tau \preceq \rho \dashv \text{seq}(D, S)} \quad [\text{ALIGN-PRODUCT-MATCHER}]$$

$$\frac{\text{demandClass}(S\tau, S\rho) = \text{slotTupleLift} \quad \text{productSlotDuals?}(S\tau) = \text{some } \bar{\eta} \quad S\rho = \text{MatcherSlot } \kappa v \quad \text{ExactCapMGU}_\Omega((\bar{\eta}.\bar{\kappa}), \kappa, C) \quad \text{ExactPairedMGU}_\Omega(C(\bar{\eta}.\bar{\tau}), C v, T)}{S; \Omega \vdash \tau \preceq \rho \dashv \text{seq}(T, \text{seq}((C, \text{id}), S))} \quad [\text{ALIGN-SLOT-TUPLE}]$$

$$\frac{S\tau = \text{Matcher } \kappa_p \tau_p \quad S\rho = \text{MatcherSlot } \kappa_c \tau_c \quad \text{OneWayDelta}_\Omega(\kappa_p, \tau_p, \kappa_c, \tau_c, D)}{S; \Omega \vdash \tau \preceq \rho \dashv \text{seq}(D, S)} \quad [\text{ALIGN-MATCHER-SLOT}]$$

$$\frac{S\tau = \text{MatcherSlot } \kappa_1 \tau_1 \quad S\rho = \text{MatcherSlot } \kappa_2 \tau_2 \quad \text{ExactCapMGU}_\Omega(\kappa_1, \kappa_2, C) \quad \text{ExactPairedMGU}_\Omega(C\tau_1, C\tau_2, T)}{S; \Omega \vdash \tau \preceq \rho \dashv \text{seq}(T, \text{seq}((C, \text{id}), S))} \quad [\text{ALIGN-SLOT-SLOT}]$$

$$\frac{\text{demandClass}(S\tau, S\rho) = \text{ordinary} \quad S; \Omega \vdash \tau \doteq \rho \dashv S'}{S; \Omega \vdash \tau \preceq \rho \dashv S'} \quad [\text{ALIGN-ORDINARY}]$$

product-of-matchers branch は whole-product matcher の生成と matcher-to-slot 消費を一つの checking cut で行う.

non-ordinary class は resolved expected head が slot の場合だけであり, matcher-headed expected type は ordinary alignment へ退化する.

4.4 唯一の checking 規則

expected type は synthesis の入力にならず, synthesis が返した cut の S_1, Ω_1 でだけ alignment を選ぶ.

$$\frac{q; S; \Omega; \Gamma \vdash e \Rightarrow \tau \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \tau \preceq \rho \dashv S'}{q; S; \Omega; \Gamma \vdash e \Leftarrow \rho \dashv q_1; S'; \Omega_1}$$

[DEMAND-DIRECTED-CHECK]

$S_1\rho$ が未解決 target variable である場合, slot head はまだ観測できないため, この規則の alignment は ordinary equality に限られる. selector は raw synthesis 結果を保持し, 後続 use を見越して variable を slot へ構造化しない. list 規則が子の出力 state を source 順に次の子へ渡すため, 先行 cut が expected head を slot へ解決したかどうかは後続 cut の branch 選択に影響する. この chronological な順序は判断の一部であり, 子の交換則は仮定しない.

5 式の synthesis 規則

本章は, 第3章の instantiation / generalization と第4章の alignment を使い, 各 expression constructor を三状態の chronological transition として定義する. 規則は局所的な synthesis 結果を返し, 外側の checking cut だけが coercion を選ぶ. let と matcher recursion で局所事実が root 終端まで安定しない箇所は terminal audit へ送る.

α_{q_τ} は supply q が指す次の target variable である. list 判断は要素を左から右へ処理し, q, S, Ω の三状態を順に渡す. instance の上付き ren は fresh capability binder image を renameOnly, str は structuralFlexible として ledger へ追加することを表す.

5.1 変数・関数・適用

変数 lookup は prevailing substitution を context に適用してから scheme を fresh instantiate する. lambda domain は fresh metavariable とする. application は function synthesis, function type との ordinary alignment, argument checking の順に進む.

$$\frac{(ST)(x) = \sigma \quad \text{Inst}_q^{\text{ren}}(\Omega, \sigma) = (\tau, q', \Omega')}{q; S; \Omega; \Gamma \vdash x \Rightarrow \tau \dashv q'; S; \Omega'} \quad \frac{q + \langle 0, 1 \rangle; S; \Omega; \Gamma, x : \text{mono } \alpha_{q_\tau} \vdash e \Rightarrow \tau \dashv q'; S'; \Omega'}{q; S; \Omega; \Gamma \vdash \lambda x. e \Rightarrow \alpha_{q_\tau} \rightarrow \tau \dashv q'; S'; \Omega'}$$

[DEMAND-DIRECTED-VAR]

[DEMAND-DIRECTED-LAM]

$$\frac{q; S; \Omega; \Gamma \vdash e_1 \Rightarrow \tau_f \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \tau_f \doteq \alpha_{(q_1)_\tau} \rightarrow \alpha_{(q_1)_{\tau+1}} \dashv S_2 \quad q_1 + \langle 0, 2 \rangle; S_2; \Omega_1; \Gamma \vdash e_2 \Leftarrow \alpha_{(q_1)_\tau} \dashv q_2; S_3; \Omega_2}{q; S; \Omega; \Gamma \vdash e_1 e_2 \Rightarrow \alpha_{(q_1)_{\tau+1}} \dashv q_2; S_3; \Omega_2}$$

[DEMAND-DIRECTED-APP]

function domain が slot に解決された場合, argument の構文に関係なく, 最後の checking cut で matcher-to-slot coercion を選べる. したがって未解決な lambda domain を複数箇所でも共有すると, 左から右の処理順が受理を左右しうる. 既知の slot domain を要求する use が先なら後続 argument を coerce できるが, 通常適用が先に raw matcher domain を確定すると, 後続の slot 要求とは ordinary equality で一致しない. これは no-guess と状態の chronological threading から従う意図された仕様である. 具体的な二つの closed program $e_{\text{demand-first}}$ と $e_{\text{ordinary-first}}$ について, Lean は

$$(\exists \tau. \text{SourceTyping}(\widehat{\Sigma}_{\text{probe}}, \emptyset, e_{\text{demand-first}}, \tau)) \wedge \neg(\exists \tau. \text{SourceTyping}(\widehat{\Sigma}_{\text{probe}}, \emptyset, e_{\text{ordinary-first}}, \tau))$$

を検査する。pre-scan, 子の並べ替え, checking obligation の遅延, source-order permutation invariance は現行規則に含めない。

変数規則の instance は canonical scheme の finite opening から計算される。state erasure では, root の終端 substitution が幕等であることと opening transport を使い, この instance が終端 context の lookup と一致することを代数的に導く。variable node 自体に追加の terminal audit field はない。binder 名の masking や capture 条件も必要ない。

5.2 リテラル・タプル・コンストラクタ

$$\frac{}{q; S; \Omega; \Gamma \vdash n \Rightarrow \text{Int} \dashv q; S; \Omega} \quad \frac{q; S; \Omega; \Gamma \vdash \bar{e} \Rightarrow \bar{\tau} \dashv q'; S'; \Omega'}{q; S; \Omega; \Gamma \vdash (\bar{e}) \Rightarrow (\bar{\tau}) \dashv q'; S'; \Omega'} \\ \text{[DEMAND-DIRECTED-LIT]} \quad \text{[DEMAND-DIRECTED-TUPLE]}$$

$$\frac{\text{Inst}_q^{\text{str}}(\Omega, \widehat{\Sigma}_D(C)) = (\bar{\rho} \rightarrow \rho, q_0, \Omega_0) \quad q_0; S; \Omega_0; \Gamma \vdash \bar{e} \Leftarrow \bar{\rho} \dashv q'; S'; \Omega_1 \quad \Omega' = \text{freezeExport}(S', \rho, \Omega_1)}{q; S; \Omega; \Gamma \vdash C \bar{e} \Rightarrow \rho \dashv q'; S'; \Omega'} \\ \text{[DEMAND-DIRECTED-CON]}$$

$$\frac{\text{Inst}_q^{\text{str}}(\Omega, \widehat{\Sigma}_{\text{prim}}(op)) = (\bar{\rho} \rightarrow \rho, q_0, \Omega_0) \quad q_0; S; \Omega_0; \Gamma \vdash \bar{e} \Leftarrow \bar{\rho} \dashv q'; S'; \Omega_1 \quad \Omega' = \text{freezeExport}(S', \rho, \Omega_1)}{q; S; \Omega; \Gamma \vdash op(\bar{e}) \Rightarrow \rho \dashv q'; S'; \Omega'} \\ \text{[DEMAND-DIRECTED-PRIM]}$$

freezeExport は exported result に残る structural leaf だけを rename-only にする。

5.3 Let と再帰

$$\frac{\sigma = \text{Gen}(\widehat{\Sigma}, S_1 \Gamma, S_1 \tau_1) \quad q; S; \Omega; \Gamma \vdash e_1 \Rightarrow \tau_1 \dashv q_1; S_1; \Omega_1 \quad q_1; S_1; \Omega_1; \Gamma, x : \sigma \vdash e_2 \Rightarrow \tau_2 \dashv q'; S'; \Omega' \quad S' \sigma = \text{Gen}(\widehat{\Sigma}, S' \Gamma, S' \tau_1)}{q; S; \Omega; \Gamma \vdash \text{let } x = e_1 \text{ in } e_2 \Rightarrow \tau_2 \dashv q'; S'; \Omega'} \\ \text{[DEMAND-DIRECTED-LET]}$$

$$\frac{f \neq x \quad \text{DirectSelf}(f, e) \quad \text{NonMatcherBody}(e) \quad q + \langle 0, 2 \rangle; S; \Omega; \Gamma, f : \text{mono}(\alpha_{q_\tau} \rightarrow \alpha_{q_\tau+1}), x : \text{mono} \alpha_{q_\tau} \vdash e \Rightarrow \rho \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \rho \doteq \alpha_{q_\tau+1} \dashv S'}{q; S; \Omega; \Gamma \vdash \text{fix } f \ x.e \Rightarrow \alpha_{q_\tau} \rightarrow \alpha_{q_\tau+1} \dashv q_1; S'; \Omega_1} \\ \text{[DEMAND-DIRECTED-FIX]}$$

let の局所安定性だけでは, 外側の後続 solve を越えた公開終端での一般化一致は表せない。そこで terminal audit は, root の終端 context と終端 value type から Gen を再計算し, body で使った scheme への終端作用と一致する事実を保持する。相互 erasure はこの事実を使って value-flow instance を再構成する。第 2 章の不一致例は任意の suffix に対して局所 cut だけを考えた例であり, その suffix へ到達する source program ではない。実際の raw traversal が生成するすべての後続 solve について pending let scheme が保存されるという終端安定性は, 別の全走査不変条件として未証明である。

DirectSelf(f, e) は shadowing-aware な構文条件であり, f の自由出現を application spine の head に限る。matcher literal を body に持つ recursion は次章の専用 placeholder 規則を使う。

5.4 標準 matcher producer

$$\frac{}{q; S; \Omega; \Gamma \vdash \text{something} \Rightarrow \text{Matcher Any } \alpha_{q_\tau} \dashv q + \langle 0, 1 \rangle; S; \Omega}$$

[DEMAND-DIRECTED-SOMETHING]

`something` 自身は matcher producer である。slot が必要かどうかはこの規則では決めず、外側の checking cut が expected head を見て決める。

6 Pattern と matcher の規則

本章は、expression checking を pattern, matcher clause, matcher literal へ拡張する。pattern は target だけでなく capability demand と binding context を合成するため、expression より多くの相互再帰 family を必要とする。各 hole の next matcher は第 4 章と同じ slot checking を使い、matcher finalization と pattern constructor compatibility の非局所事実は terminal audit へ渡す。この構造が第 7 章の相互 erasure と受理完全性の pattern 側の依存になる。

6.1 ユーザーパターンの synthesis

ユーザーパターン判断を

$$q; S; \Omega; \Gamma; \Phi; \Delta \vdash p \Rightarrow \eta; \Delta' \dashv q'; S'; \Omega'$$

と書く。pattern dual $\eta = \kappa \triangleright \tau$ と binding context を同時に合成し、 Δ を左から右へ延長する。fresh pattern capability は consumer の局所変数なので `structuralFlexible` として ledger に追加する。

$$\frac{x \notin \text{dom}(\Delta) \quad \Omega' = \Omega[\chi_{q_\kappa} \mapsto \text{structuralFlexible}]}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash \$x \Rightarrow \chi_{q_\kappa} \triangleright \alpha_{q_\tau}; \Delta, x : \alpha_{q_\tau} \dashv q + \langle 1, 1 \rangle; S; \Omega'}$$

[DEMAND-DIRECTED-PAT-VAR]

$$\frac{\Omega' = \Omega[\chi_{q_\kappa} \mapsto \text{structuralFlexible}]}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash _ \Rightarrow \chi_{q_\kappa} \triangleright \alpha_{q_\tau}; \Delta \dashv q + \langle 1, 1 \rangle; S; \Omega'}$$

[DEMAND-DIRECTED-PAT-WILD]

$$\frac{q; S; \Omega; \text{mono } \Delta, \Gamma \vdash e \Rightarrow \tau \dashv q_1; S_1; \Omega_1 \quad \Omega' = \Omega_1[\chi_{(q_1)_\kappa} \mapsto \text{structuralFlexible}]}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash \#e \Rightarrow \chi_{(q_1)_\kappa} \triangleright \tau; \Delta \dashv q_1 + \langle 1, 0 \rangle; S_1; \Omega'}$$

[DEMAND-DIRECTED-PAT-VALUE]

$$\frac{\Phi(x) = \kappa \triangleright \tau}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash \tilde{x} \Rightarrow \kappa \triangleright \tau; \Delta \dashv q; S; \Omega}$$

[DEMAND-DIRECTED-PAT-EMBED]

tuple と constructor は子を先に合成する。constructor scheme は structural instance を作り、field target alignment

と capability projection の後, result に残る structural leaf を freeze する.

$$\frac{q; S; \Omega; \Gamma; \Phi; \Delta \vdash \bar{p} \Rightarrow \bar{\eta}; \Delta' \dashv q'; S'; \Omega'}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash (\bar{p}) \Rightarrow (\bar{\eta}.\bar{\kappa}) \triangleright (\bar{\eta}.\bar{\tau}); \Delta' \dashv q'; S'; \Omega'}$$

[DEMAND-DIRECTED-PAT-TUPLE]

$$\frac{\text{Inst}_q^{\text{str}}(\Omega, \widehat{\Sigma}_P(c)) = (\bar{p} \Rightarrow_P \rho, q_0, \Omega_0) \quad q_0; S; \Omega_0; \Gamma; \Phi; \Delta \vdash \bar{p} \Rightarrow \bar{\eta}; \Delta' \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \bar{\eta}.\bar{\tau} \doteq \bar{p} \dashv S_2}{\frac{q_1; S_2; \Omega_1 \vdash_c \bar{\eta}.\bar{\kappa} \Rightarrow \kappa \dashv q_2; S_3; \Omega_2 \quad \text{CapCompatible}_{\widehat{\Sigma}}(c; S_3\bar{\eta}.\bar{\kappa}, S_3\kappa) \quad \Omega' = \text{freezeExport}(S_3, \kappa \triangleright \rho, \Omega_2)}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash c\bar{p} \Rightarrow \kappa \triangleright \rho; \Delta' \dashv q_2; S_3; \Omega'}}$$

[DEMAND-DIRECTED-PAT-CON]

ここで $\vdash_c$ は signature entry が定める constructor capability projection である. 規則中の **CapCompatible** はこの局所 cut での検査である. 公開終端まで後続 solve が続く場合に備え, terminal audit は終端 substitution を dual 列と result capability へ適用した後の **CapCompatible** を再検査する. state erasure はこの終端事実を使うため, 局所 capability が以後不変であることを仮定しない.

p_1 & p_2 は左の bindings を右へ渡す. $p_1 \mid p_2$ は同じ入力 Δ から両 alternative を調べ, dual と binder 名・型を最後に align する.

$$\frac{q; S; \Omega; \Gamma; \Phi; \Delta \vdash p_1 \Rightarrow \eta_1; \Delta_1 \dashv q_1; S_1; \Omega_1 \quad q_1; S_1; \Omega_1; \Gamma; \Phi; \Delta_1 \vdash p_2 \Rightarrow \eta_2; \Delta_2 \dashv q_2; S_2; \Omega_2 \quad S_2; \Omega_2 \vdash \eta_1 \doteq \eta_2 \dashv S'}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash p_1 \& p_2 \Rightarrow \eta_1; \Delta_2 \dashv q_2; S'; \Omega_2}$$

[DEMAND-DIRECTED-PAT-AND]

$$\frac{q; S; \Omega; \Gamma; \Phi; \Delta \vdash p_1 \Rightarrow \eta_1; \Delta_1 \dashv q_1; S_1; \Omega_1 \quad q_1; S_1; \Omega_1; \Gamma; \Phi; \Delta \vdash p_2 \Rightarrow \eta_2; \Delta_2 \dashv q_2; S_2; \Omega_2 \quad S_2; \Omega_2 \vdash \eta_1 \doteq \eta_2 \dashv S_3 \quad S_3; \Omega_2 \vdash \Delta_1 \doteq \Delta_2 \dashv S'}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash p_1 \mid p_2 \Rightarrow \eta_1; \Delta_1 \dashv q_2; S'; \Omega_2}$$

[DEMAND-DIRECTED-PAT-OR]

pattern-function scheme の capability binder は instance 時点から rename-only である.

$$\frac{\text{InstDual}_q^{\text{ren}}(\Omega, \widehat{\Sigma}_F(f)) = (\bar{\eta} \Rightarrow \eta, q_0, \Omega_0) \quad q_0; S; \Omega_0; \Gamma; \Phi; \Delta \vdash \bar{p} \Rightarrow \bar{\eta}'; \Delta' \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \bar{\eta}' \doteq \bar{\eta} \dashv S'}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash f\bar{p} \Rightarrow \eta; \Delta' \dashv q_1; S'; \Omega_1}$$

[DEMAND-DIRECTED-PAT-APP]

6.2 MatchAll の型付け

matchAll は target を合成し, pattern target と align した後, matcher expression を slot に対して check する. 第四 premise はこの規則が直接導入する唯一の slot demand である. matcher expression が matcher literal なら, その内部の各 next matcher も別の checking cut を持つ.

$$\frac{q; S; \Omega; \Gamma \vdash e_t \Rightarrow \tau_t \dashv q_1; S_1; \Omega_1 \quad q_1; S_1; \Omega_1; \Gamma; \emptyset; \emptyset \vdash p \Rightarrow \kappa_p \triangleright \tau_p; \Delta \dashv q_2; S_2; \Omega_2 \quad S_2; \Omega_2 \vdash \tau_p \doteq \tau_t \dashv S_3}{\frac{q_2; S_3; \Omega_2; \Gamma \vdash e_m \Leftarrow \text{MatcherSlot } \kappa_p \tau_t \dashv q_3; S_4; \Omega_3 \quad q_3; S_4; \Omega_3; \text{mono } \Delta, \Gamma \vdash e \Rightarrow \tau_r \dashv q'; S'; \Omega'}{q; S; \Omega; \Gamma \vdash \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e \Rightarrow \text{List } \tau_r \dashv q'; S'; \Omega'}}$$

[DEMAND-DIRECTED-MATCHALL]

6.3 Primitive pattern と data pattern

primitive pattern 判断は shared matcher target に対して hole dual 列と bindings を返す. hole だけが fresh consumer capability を生成する.

$$\begin{array}{c}
\frac{\Omega' = \Omega[\chi_{q_\kappa} \mapsto \text{structuralFlexible}]}{q; S; \Omega \vdash \$ \Leftarrow \tau \Rightarrow [\chi_{q_\kappa} \triangleright \tau]; \emptyset \dashv q + \langle 1, 0 \rangle; S; \Omega'} \quad \frac{}{q; S; \Omega \vdash _ \Leftarrow \tau \Rightarrow []; \emptyset \dashv q; S; \Omega} \\
\text{[DEMAND-DIRECTED-PP-HOLE]} \quad \text{[DEMAND-DIRECTED-PP-WILD]} \\
\\
\frac{}{q; S; \Omega \vdash \# \$ x \Leftarrow \tau \Rightarrow []; x : \tau \dashv q; S; \Omega} \\
\text{[DEMAND-DIRECTED-PP-VALUE]} \\
\\
\frac{\text{Inst}_q^{\text{str}}(\Omega, \widehat{\Sigma}_P(c)) = (\bar{\rho} \Rightarrow_P \rho, q_0, \Omega_0) \quad S; \Omega_0 \vdash \rho \doteq \tau \dashv S_1 \quad q_0; S_1; \Omega_0 \vdash \overline{pp} \Leftarrow \bar{\rho} \Rightarrow \bar{\eta}; \Delta \dashv q'; S'; \Omega_1}{q; S; \Omega \vdash c \overline{pp} \Leftarrow \tau \Rightarrow \bar{\eta}; \Delta \dashv q'; S'; \Omega_1} \\
\text{[DEMAND-DIRECTED-PP-CON]} \\
\\
\frac{\text{freshTargets}(n, q) = (\bar{\alpha}, q_0) \quad S; \Omega \vdash (\bar{\alpha}) \doteq \tau \dashv S_1 \quad q_0; S_1; \Omega \vdash \overline{pp} \Leftarrow \bar{\alpha} \Rightarrow \bar{\eta}; \Delta \dashv q'; S'; \Omega'}{q; S; \Omega \vdash (\overline{pp}) \Leftarrow \tau \Rightarrow \bar{\eta}; \Delta \dashv q'; S'; \Omega'} \\
\text{[DEMAND-DIRECTED-PP-TUPLE]}
\end{array}$$

data pattern 判断は arm の bindings を返す.

$$\begin{array}{c}
\frac{}{q; S; \Omega \vdash \$ x \Leftarrow \tau \Rightarrow x : \tau \dashv q; S; \Omega} \quad \frac{}{q; S; \Omega \vdash _ \Leftarrow \tau \Rightarrow \emptyset \dashv q; S; \Omega} \\
\text{[DEMAND-DIRECTED-DP-VAR]} \quad \text{[DEMAND-DIRECTED-DP-WILD]} \\
\\
\frac{\text{Inst}_q^{\text{str}}(\Omega, \widehat{\Sigma}_D(C)) = (\bar{\rho} \rightarrow \rho, q_0, \Omega_0) \quad S; \Omega_0 \vdash \rho \doteq \tau \dashv S_1 \quad q_0; S_1; \Omega_0 \vdash \overline{dp} \Leftarrow \bar{\rho} \Rightarrow \Delta \dashv q'; S'; \Omega'}{q; S; \Omega \vdash C \overline{dp} \Leftarrow \tau \Rightarrow \Delta \dashv q'; S'; \Omega'} \\
\text{[DEMAND-DIRECTED-DP-CON]} \\
\\
\frac{\text{freshTargets}(n, q) = (\bar{\alpha}, q_0) \quad S; \Omega \vdash (\bar{\alpha}) \doteq \tau \dashv S_1 \quad q_0; S_1; \Omega \vdash \overline{dp} \Leftarrow \bar{\alpha} \Rightarrow \Delta \dashv q'; S'; \Omega'}{q; S; \Omega \vdash (\overline{dp}) \Leftarrow \tau \Rightarrow \Delta \dashv q'; S'; \Omega'} \\
\text{[DEMAND-DIRECTED-DP-TUPLE]}
\end{array}$$

6.4 Clause と matcher literal

一つの clause は primitive pattern, next-matcher 列, arm 列の順に処理する. 各 next matcher は対応する hole slot に対して同じ checking 判断を使う.

$$\frac{q; S; \Omega \vdash pp \Leftarrow \tau \Rightarrow \bar{\eta}; \Delta_{pp} \dashv q_1; S_1; \Omega_1 \quad \text{decomposeME}(M, |\bar{\eta}|) = \text{some } \bar{M} \quad q_1; S_1; \Omega_1; \Gamma \vdash \bar{M} \Leftarrow \text{MatcherSlot } \eta.\kappa \eta.\tau \dashv q_2; S_2; \Omega_2 \quad q_2; S_2; \Omega_2; \Gamma; \Delta_{pp} \vdash \bar{a} \Leftarrow \tau; \text{List prod}(\bar{\eta}.\bar{\tau}) \dashv q'; S'; \Omega'}{q; S; \Omega; \Gamma \vdash pp \text{ as } M \text{ with } \bar{a} \Leftarrow \tau \Rightarrow \bar{\eta} \dashv q'; S'; \Omega'} \\
\text{[DEMAND-DIRECTED-CLAUSE]}$$

matcher literal は全 clause で一つの fresh target を共有する. 終端 substitution で hole dual を解決して evidence を再計算し, shape, catch-all order, binder 線形性, arm exhaustiveness, coverage を検査する. 最後に, final capability に現れる matcher-owned explicit ledger key の structural leaf だけを freeze する. active context の top-level slot demand または direct-self の引数 slot から借りた leaf を borrowed leaf と呼び, これは matcher 自身が生成した

interface ではないため新規 freeze 対象から除く。

$$\frac{\begin{array}{l} q + \langle 0, 1 \rangle; S; \Omega; \Gamma \vdash cls \Leftarrow \alpha_{q_r} \Rightarrow \bar{\eta} \dashv q'; S'; \Omega_1 \\ \text{collectClauseEvidence}_{\bar{\Sigma}}(cls, \text{terminalHoleCaps}(S', \bar{\eta})) = \text{some } \bar{e}v \quad \text{inferShape}_{\bar{\Sigma}}(\bar{e}v) = \text{some } \kappa \\ \text{clauseCapsListCheck}_{\bar{\Sigma}}(\kappa, cls, \text{terminalHoleCaps}(S', \bar{\eta})) \quad \text{CatchAllLast}(cls) \quad \text{MatcherBindersCheck}(cls) \\ \text{ArmExhaustive}_{\bar{\Sigma}_D}(cls, S' \alpha_{q_r}) \quad \text{CoverageOK}_{\bar{\Sigma}}(cls, \kappa) \quad \Omega' = \text{freezeMatcher}(S', \kappa, \Omega_1) \end{array}}{q; S; \Omega; \Gamma \vdash \text{matcher } cls \Rightarrow \text{Matcher } \kappa \alpha_{q_r} \dashv q'; S'; \Omega'}$$

[DEMAND-DIRECTED-MATCHER]

上の evidence は matcher 自身の局所終端 S' で計算される。matcher-owned で freeze された leaf は root 終端まで rename-only であるが、borrowed leaf は外側の後続 solve で構造的に特殊化され得る。terminal audit は root 終端で hole capability を解決し直し、evidence 収集、shape、clause capability、arm exhaustiveness、coverage を再検査する。matcher の相互 erasure はこの正規化済み事実から `TypingInvariant` constructor を作り、局所の solver trace や MGU を `typing invariant` へ残さない。

現行の終端再検査は borrowed leaf の由来を保持せず、その leaf が観察不能な constructor へ特殊化された場合にも再帰的な shape を要求する。第 2 章の閉じた反例では、matcher 自身は一要素 tuple だけを分解して要素を引数 matcher へ委譲するが、borrowed leaf が後から K へ特殊化されるため、9 検査中 `traceFinalizationSuffixCheck` だけが失敗する。従って「borrowed leaf を freeze しない」という局所規則と「root で final shape 全体を再計算する」という現行 audit の組合せには、実際の source traversal で到達する terminal audit による受理損失がある。

修正候補は、matcher 自身が収集した evidence と、引数 slot から完全な値として借りた leaf を区別する情報を terminal finalization まで運び、後者の内部だけは再帰的に観察しない方法である。単に borrowed leaf も freeze すると引数 matcher に応じた正当な特殊化を失う。逆にすべての opaque leaf を許すと、matcher 自身が opaque な内部構造を調べる場合まで通すため、どちらもこの区別の代わりにはならない。

matcher literal を body に持つ recursion は、clause skeleton から placeholder を作る専用規則を使う。placeholder range の capability key は matcher 開始前に structural-flexible として ledger へ追加される。

$$\frac{\begin{array}{l} f \neq x \quad \text{DirectSelf}(f, \text{matcher } cls) \\ \text{fixMatcherPlaceholderSupply}_{\bar{\Sigma}}(cls, q) = \text{some } (\tau_d, \tau_c, q_0) \quad \Omega_0 = \text{markCapRange}(q, q_0, \Omega) \\ q_0; S; \Omega_0; \Gamma, f : \text{mono } (\tau_d \rightarrow \tau_c), x : \text{mono } \tau_d \vdash \text{matcher } cls \Rightarrow \rho \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \rho \doteq \tau_c \dashv S' \end{array}}{q; S; \Omega; \Gamma \vdash \text{fix } f \ x.(\text{matcher } cls) \Rightarrow \tau_d \rightarrow \tau_c \dashv q_1; S'; \Omega_1}$$

[DEMAND-DIRECTED-FIX-MATCHER]

`NonMatcherBody` により通常の `DEMAND-DIRECTED-FIX` と排他的である。

7 機械化された性質と接続

本章は、第 3–6 章の規則を公開定理へ接続する。証明は「局所 solver の代数」「全 traversal への持上げ」「source / executable / runtime 境界」の三層に分かれる。個別の Lean module、補題、回帰の索引は `../docs/details.md` に分離する。

7.1 公開定理の依存関係

公開結果は互いに独立な大定理ではなく、共通の局所不変量を次のように消費する。

公開結果	直接使う証明境界	その依存が必要な理由
$\text{infer} \Rightarrow \text{SourceTyping}$	executable solver correspondence, trace history, validator soundness	成功 run と同じ cut 列を持つ Origin derivation と terminal audit を作る
$\text{SourceTyping} \Rightarrow \text{infer}$	exact solve completeness, state bisimulation, audit から event coverage への輸送	関係的 derivation を同じ割当順序の成功 run として再生する
$\text{SourceTyping} \Rightarrow \text{TypingInvariant}$	chronological factorization, solved form, terminal audit, scheme closedness	局所状態を一つの root 終端へ正規化してから消去する
SourceTyping.safe	state erasure, FrozenSigWF, CoreSafety	source 型を保ったまま runtime 側の安全性 package へ接続する
受理同値と決定可能性	executable soundness と acceptance completeness	source typability を有限の公開推論で特徴付ける
target 一意性	acceptance completeness, 実行 run の決定性, 局所 renaming	二導出を同じ実行 target へ写し, MGU の向きを隠す
closed DM 保守性	closed state erasure, type / context erasure, generalization 保存	監査済み source 導出を pattern-free DM 導出へ写す

FrozenSigWF は source-facing な唯一の global signature 条件である。受理完全性には scheme closedness と canonical arm checker, 安全性には scheme closedness と動的整合性を供給する。低レベル補題は必要な field を明示してよいが, 公開定理は caller にそれらを別々に渡させない。

7.2 局所代数を全判断へ持ち上げる

Slot demand

$S; \Omega \vdash \tau \preceq \rho \dashv S'$ が ordinary equality でないなら, resolved expected type は必ず slot-headed である。

$$S; \Omega \vdash \tau \preceq \rho \dashv S' \implies (S; \Omega \vdash \tau \doteq \rho \dashv S') \vee \exists \kappa, v. S\rho = \text{MatcherSlot } \kappa v.$$

特に $S\rho$ が matcher-headed なら ordinary alignment に限られる。これにより「coercion は slot demand を観測した checking cut で一度だけ」が実装上の慣例ではなく judgment の定理になる。

State extension と boundedness

全 raw demand-directed family は supply を単調に進め, 出力 substitution を入力 substitution と局所 delta の chronological replay へ因子化する。対応する Origin family は ledger の既存 entry を保ち, fresh supply の範囲内だけを追加・freeze する。公開型, pattern dual, bindings, hole ledger に現れる flexible variable は終端 supply で有界である。これにより子の出力を次の子へ渡す規則と, 実行 traversal の割当順序を一致させる。

Solved form

capability-only, target-only, paired, matcher / slot の二段階 solve, one-way matching の各局所操作は, 入力 substitution の冪等性を保存する。これを traversal 順に合成し, alignment と expression, checking, pattern, arm, clause を含む全 14 raw family について

$$S.\text{Idempotent} \implies S'.\text{Idempotent}$$

を得る。公開判断は恒等 substitution から始まるため, root 終端は無条件に solved form である。

No guess と freeze

exact MGU は constraint 外で恒等, constraint range 内, solved form である。Origin admissibility は rigid key を固定し, rename-only key の structural strengthening を拒否する。constructor / fresh consumer の structural variable は局所 solve を許すが, export または matcher finalization を越えて公開 payload に残る matcher-owned producer

leaf は rename-only になる。active context の top-level slot demand または direct-self の引数 slot から借りた leaf は、matcher が生成した interface ではないため新規 freeze 対象から除く。

この境界がなければ、量化 producer capability を後続 solve が Any へ強化する program や、let-generalized capability を強化する program を受理してしまう。現行 SourceTyping では、対応する delta の拒否と program 全体の導出不能性を negative regression として閉じている。一方、or-pattern, delegating matcher, let-polymorphic producer には public Origin certificate がある。

7.3 実行可能推論から SourceTyping へ

公開推論は停止する raw traversal と有限の fail-closed validator の合成である。

$$\text{infer}(\hat{\Sigma}, \Gamma, e) = \text{bind}(\text{inferRaw}(\hat{\Sigma}, \Gamma, e), \lambda r. \text{if } \text{wBridgeCheck}(\hat{\Sigma}, r) \text{ then some } r \text{ else none}).$$

raw run から source derivation を作るには、次の三段階が必要である。

1. executable solver の成功を、ledger-admissible な exact relational solve へ写す。
2. expression, checking, pattern, matcher, arm, clause の traversal を、同じ q, S, Ω を持つ raw derivation と Origin certificate へ相互再構成する。
3. append-only history で全局所 cut を一つの root 終端へ接続し、validator soundness から let, matcher, pattern constructor の terminal audit を構成する。

従って

$$\frac{\text{infer}(\hat{\Sigma}, \Gamma, e) = \text{some } r}{\text{SourceTyping}(\hat{\Sigma}, \Gamma, e, r.\text{resolvedTarget})} \quad (\text{INFERENCE-SOUNDNESS})$$

を得る。caller-supplied bridge, history, InferenceInputWF は premise に残らない。この経路は TypingInvariant を介さず、唯一の source typing を直接再構成する。

これとは独立に、成功 run の proof-relevant reconstruction から $\text{TypingInvariant}(\hat{\Sigma}, S_r\Gamma, e, r.\text{resolvedTarget})$ を得る内部経路もある。これは動的メタ理論へ直接接続するための経路であり、一般 context について二つの経路を同一視しない。

7.4 SourceTyping から実行可能推論へ

逆向きは、Origin tree と terminal audit を同時に再帰し、関係的 derivation を成功 run として再生する。exact solve witness と executable solver result は、raw metavariable 名の構文的一致ではなく、互いに factor する idempotent state の bisimulation で対応させる。fresh allocation, context normalization, producer protection, fuel bound を全 traversal family で保存する。

各局所 run は成功等式と validator event coverage を返す。ordinary event は規則自身から得る。let generalization, matcher finalization, pattern-constructor compatibility は terminal audit から得て、demand-directed 側と実行側の operand を bisimulation 越しに対応させる。root で全 event を有限の wBridgeCheck へ射影する。FrozenSigWF はこの輸送に必要な scheme closedness と canonical arm checker を供給する。

実際の公開定理は一般 context について

$$\text{SourceTyping}(\hat{\Sigma}, \Gamma, e, \tau) \implies \text{FrozenSigWF}(\hat{\Sigma}) \implies \text{infer}(\hat{\Sigma}, \Gamma, e) \neq \text{none} \quad (\text{ACCEPTANCE-COMPLETENESS})$$

である。raw visibility, freeze compatibility, solver success, validator bridge は caller premise に残らない。ただしこれは validator 単体が任意の raw run を受理するという主張ではなく、terminal-audited SourceTyping から再構成した run に対する受理完全性である。言い換えると、結論の「completeness」は audit を定義に含む唯一の source judgment

に相対的であり、raw demand-directed derivation に対する audit の完全性や、audit と独立な宣言的型付けに対する完全性ではない。

この差は空ではない。整形形式な signature と閉じた program について、inferRaw と対応する DemandSynthRun は存在する一方、9 個の terminal check が

(true, true, true, true, true, true, true, false, true)

となる回帰がある。失敗するのは traceFinalizationSuffixCheck だけであり、公開 infer は none を返す。原因は、引数 matcher から借りた slot の capability variable が局所 finalization 後に観察不能な K へ特殊化され、終端検査が委譲先の K まで再帰的に観察しようとするものである。従ってこの定理から「raw 成功は必ず audit を通る」とは導けない。現在の安全性定理も audit を含む SourceTyping を前提とするため、この program の独立した安全性や、安全な program 全体に対する terminal audit による受理損失の正確な範囲は未証明である。

recursive list matcher, multiset matcher, pattern constructor を含む matchAll を、外部から paired root を渡さずこの公開定理へ到達する end-to-end regression として機械検査している。

7.5 Damas–Milner 断片の実行可能受理

capability binder を持たない pattern-free な一 sort judgment $\text{DM.Typeing}(\Gamma, e, \tau)$ については、constructor ごとの Algorithm W replay を別の相互帰納で構成する。variable の canonical opening, lambda / application の fresh target, chronological list, let generalization, direct-self fix を、一つの paired residual と retired-cut 監査の下で接続する。公開 root では初期 substitution, frontier, pending cut, provenance surface を canonical な空状態に固定するため、caller-supplied な run certificate は残らない。得られる定理は

$$\frac{\text{FrozenSigWF}(\hat{\Sigma}) \quad \text{DM.Typeing}(\Gamma, e, \tau)}{\text{inferenceSucceeds}(\hat{\Sigma}, \text{emb}(\Gamma), e) = \text{true}} \quad (\text{DM-ACCEPTANCE})$$

である。 τ は結論に現れない。DM derivation が選ぶ型は公開推論器の principal target の instance であり得るため、推論返値と $\text{emb}(\tau)$ の構文的一致は要求しない。逆方向の closed source fragment から DM への消去と合わせると、その fragment の source typing から公開推論成功まで到達するが、型の一致は依然として principality 層に分離される。

7.6 State erasure と公開安全性

SourceTyping から内部 invariant を作るとき、局所 cut の状態をそのまま runtime typing へ持ち込まない。各局所出力から一つの root 終端への StateFactorization, root substitution の solved form, terminal audit を使い、全 14 Origin family を同じ終端へ正規化してから q, S, Ω を消去する。

canonical scheme の finite opening は variable lookup を終端 context へ輸送する。matcher-to-slot は終端 CapabilityDemand, slot-to-slot は終端型等式へ射影する。let, matcher, pattern constructor だけは terminal audit の三種の facts を使う。従って closed signature 上の closed program で

$$\hat{\Sigma}.\text{SchemesClosed} \implies \text{SourceTyping}(\hat{\Sigma}, \emptyset, e, \tau) \implies \text{TypingInvariant}(\hat{\Sigma}, \emptyset, e, \tau)$$

が成立する。TypingInvariant の存在を source rule の premise に置く循環はない。

FrozenSigWF はこの scheme closedness に加えて runtime signature の動的整合性を持つ。デコード全域性のため、listCtorsExclusive(list 結果へ instantiate できる data constructor は canonical な nil / cons のみ) も field であり、frozenSigWFCheck が table 全体を検査する。この条件の下で evaluation は TypingInvariant を ValueTy へ保存し、matching machine について typed state の一段保存、StepReady を前提とする局所 progress, 到達可能 state の保存、成功 branch の substitution typing が成り立つ。空 successor 列は正当な match failure であり stuck ではない。

StepReady は (a) 埋め込み式評価の収束, (b) listOfV / decodeTuple のデコード成功, (c) 成功 clause / arm への到達、を束ねた帰納述語だが、typed state では (b)(c) は仮定でなく定理である。到達 (c) は coverage を使わず CatchAllLast と

ArmExhaustive だけから、デコード成功 (b) は preservation と canonical form (listCtorsExclusive · product inversion) から放電され、構成定理

$$\text{MStateTy}(\widehat{\Sigma}, \Gamma, s, \Delta) \wedge s.S \neq [] \wedge \text{StateEvals}(\widehat{\Sigma}, \mathcal{S}_F, s) \implies \text{StepReady}(\mathcal{S}_F, s) \quad (\text{READY-OF-TYPED})$$

が成り立つ (StateEvals は先頭位置に埋め込まれた式評価の収束で、各評価は derivation-local な runtime-signature agreement を伴う)。従って公開の局所 progress MStateTy.progress_of_evals の未放電の前提は、埋め込み評価の有限導出が存在することだけに縮む。これは収束しない場合を別の発散判断で分類する主張ではない。coverage は引き続き preservation 側 (継続 atom の型付け) の条件である。

この収束前提は式層の no-stuck 定理で大域的に扱われる。evalFuel は評価 · matching · clause dispatch の全再帰を fuel で打ち切る関数的 big-step 評価器で、結果を ok(値) · timeout (fuel 切れ) · stuck(適用できる規則がない) の三値で返し、発散と詰まりを区別する。adequacy(evalFuel_ok: ok の実行は関係的 big-step 導出を再生する) がこれを既存の preservation へ接続し、fuel 上の strong induction が式 · atom · 状態 · 探索 · clause dispatch · header 照合の各層の安全性を束ねて、次を与える。

$$\frac{\text{FrozenSigWF}(\widehat{\Sigma}) \quad \forall \Gamma. \text{RuntimeSigAgrees}(\widehat{\Sigma}, \Gamma, \mathcal{S}_F) \quad \text{RuntimeSigScoped}(\mathcal{S}_F) \quad \text{TypingInvariant}(\widehat{\Sigma}, \emptyset, e, \tau) \quad \text{fv}(e) = \emptyset}{\forall n. \text{evalFuel } \mathcal{S}_F n e \neq \text{stuck}} \quad (\text{NO-STUCK})$$

(Lean: typed_never_stuck_runtime. RuntimeSigScoped は runtime 本体に自由な式変数がないことを表す。論文 1 の空表断片では patternFuns = [] を明記し、runtimeSigAgrees_nil から $\forall \Gamma. \text{RuntimeSigAgrees}(\widehat{\Sigma}, \Gamma, [])$ を構成する SourceTyping.never_stuck_paper1 を用いる。) evalFuel_eventually_ok は、有限の関係の評価があれば十分大きいすべての fuel で同じ値を返すことを示す。従って adequacy と合わせると、全 fuel での timeout は有限の関係の評価が存在しないことと一致する (typed_all_timeout_iff_no_finite_eval) が、別に定義した余帰納的な発散判断との同値は主張しない。一般の termination は非目標であり、BFS (幅優先探索) の matchAll に対する matching completeness は将来課題である。中間値の型付け · pristine 性は adequacy を介して関係的 preservation から回収する。

安全性とは別に、形式化した core が意図した Egison の機能を表すことを具体的な実行回帰でも検査する。FeatureExecutionRegression は、公開推論の成功、SourceTyping の再構成、evalFuel の正確な返値、evalFuel_ok による関係の評価、任意 fuel での no-stuck を同じ fixture 上で接続する。非線形 pattern については pair \$x #x の一致が一結果を、不一致が正常な match failure として空 list を返す。既存の有限 carrier 回帰に加えて、GeneralMultisetExecutionRegression は入力長に特化しない再帰 matcher 関数をこの型安全性の鎖へ接続し、同じ定義の実行について、空 · 一要素 · 三要素 · 重複要素の nil / cons, 入れ子 cons, join を検査する。submultisetSplits は、各出現位置を左右へ割り振る全二分割を、左側の要素数と元の添字順で列挙する primitive であり、同じ値を持つ異なる出現も別分岐として残す。入れ子 cons の結果は現在の depth-first (深さ優先) · 左から右の探索順を固定する。

論文 3 節の特殊化された三つの matcher 節も同じ回帰で個別に検査する。\$:: _ は対象全体を一度返す。#\$val :: \$ は Egison ライブラリと同じ値先行の member val tgt / deleteFirst val tgt を removeFirstChoice val tgt へコンパイルし、重複があるときも最初の出現だけを除き、値がなければ正常な不一致を返す。\$ ++ \$ は submultisetSplits で全二分割を列挙する。三節とも公開推論、SourceTyping, 正確な評価, adequacy, 全 fuel no-stuck へ接続する。

閉じた multiset 適用は element matcher に something を使い、三要素の cons, 入れ子 cons, および join を公開推論、SourceTyping, 正確な評価, adequacy, 全 fuel no-stuck へ接続する。matcher の引数 slot から借りた capability 変数は matcher 自身が生成した producer 変数ではないため、matcher 確定時の新たな freeze 対象から除く。したがって something の Any に特殊化できる一方、matcher-owned の結果 leaf と既に輸出済みの変数の保護は維持する。pattern function についても、nullary fixture は同一 program 上の全段階を接続し、parameter 付き pass は embedded parameter の展開から body 結果までの正確な実行と、同じ variable-only 境界による公開推論拒否を対で固定する。

CompositionFeatureRegression は、これらの機能を同一の matchAll で合成する P2 回帰である。一般 multiset matcher の join で左右を分け、左右の cons で要素を選び、\$x (変数 pattern) で束縛した整数を #x (value pattern) で比較する。各要素には nullary (引数を取らない) unit() pattern function も適用する。重複する二つの 1 を異なる出現位置として選ぶ成功例は、同じ値を持つ 2 分岐を正確に返す。等しい整数を持たない例は、評価器の stuck ではなく正常な

不一致として空 list を返す。両方について公開推論, `SourceTyping`, 正確な `evalFuel` 結果, adequacy による関係的評価, 任意 fuel での no-stuck を同じ fixture 上で接続する。

これらは一般の termination, 全入力 list に対する multiset matcher の機能的正当性, Egison 標準 multiset matcher の全機能, BFS completeness を主張しない。

これらを source-facing な一つの定理に束ねる。

$$\frac{\text{FrozenSigWF}(\widehat{\Sigma}) \quad \text{SourceTyping}(\widehat{\Sigma}, \emptyset, e, \tau)}{\text{SafeResult}_{\text{Source}}(\widehat{\Sigma}, e, \tau, S_F)} \quad (\text{SOURCE-SAFETY})$$

`SafeResultSource` は同じ公開型の `TypingInvariant` と `CoreSafety` を保持する。caller は内部 certificate や scheme closedness を別 premise として渡さない。

7.7 受理同値, 返値型, target 一意性

INFERENCE-SOUNDNESS と ACCEPTANCE-COMPLETENESS を合成すると, 一般 context について

$$\text{FrozenSigWF}(\widehat{\Sigma}) \implies \left(\exists \tau. \text{SourceTyping}(\widehat{\Sigma}, \Gamma, e, \tau) \iff \text{infer}(\widehat{\Sigma}, \Gamma, e) \neq \text{none} \right)$$

を得る。 $\Gamma = \emptyset$ への特殊化が annotation-freeness である。source の Expr には type-ascription constructor がないため, closed term の source typability を型 annotation なしの有限な公開推論で決定できる。同じ同値から `Decidable`($\exists \tau. \text{SourceTyping}$) も構成済みである。

型を返す入口についても

$$\text{inferType}(\widehat{\Sigma}, \Gamma, e) = \text{some } \tau \implies \text{SourceTyping}(\widehat{\Sigma}, \Gamma, e, \tau)$$

が成り立つ。任意に与えた derivation の target と返値型の構文的な一致は要求しない。二つの `SourceTyping` target τ_1, τ_2 は, acceptance completeness で同じ決定的な実行 run へ写る。各辺の state factorization から, 全 residual 二 sort meta 上で局所的に可逆な renaming R_i と共通 target v を取り出し, $R_i \tau_i = v$ を得る。

initial supply 以前の meta まで固定する強い形は一般 context では偽である。例えば $f : ?0 \rightarrow ?0, x : ?1$ の下の fx では, constraint $?1 \doteq ?0$ の exact MGU をどちら向きに取るかで公開 target は $?0$ にも $?1$ にもなる。従って一意性は全 residual meta の renaming を法として述べる。

二 sort instance preorder は, source type に現れる free capability / target metavariable を有限 scope とし, その scope 外で恒等な paired substitution により

$$\tau \preceq_{\text{inst}} \tau' \iff \exists S. \text{supp}_{\kappa}(S) \subseteq \text{fcv}(\tau) \wedge \text{supp}_{\tau}(S) \subseteq \text{ftv}(\tau) \wedge S\tau = \tau'$$

と定める。scope restriction は source への作用を変えず, 時系列合成後に元 scope へ再 restriction できるため, この関係は反射的かつ推移的である。局所 renaming とその inverse は両方向の instance witness を与える。従って

$$\begin{aligned} & \text{FrozenSigWF}(\widehat{\Sigma}) \wedge \text{inferType}(\widehat{\Sigma}, \Gamma, e) = \text{some } \tau_p \\ & \implies \text{SourceTyping}(\widehat{\Sigma}, \Gamma, e, \tau_p) \wedge \\ & \forall \tau. \text{SourceTyping}(\widehat{\Sigma}, \Gamma, e, \tau) \Rightarrow \tau_p \preceq_{\text{inst}} \tau \end{aligned}$$

を得る。空 context への特殊化が closed principality である。この証明は `TypingInvariant` 全体の principality 反例を `SourceTyping` へ流用しない。

open term では raw context の meta を rigid にせず, context と target を一つの substitution で同時に比較する。

$$(\Gamma, \tau) \preceq_{\text{ctx}} (\Gamma', \tau') \iff \exists S. S\Gamma = \Gamma' \wedge S\tau = \tau'$$

ここでも support は Γ と τ の二 sort free-variable scope の和に制限する。既存の `SourceTyping` derivation から terminal substitution による normalized context と公開 target を取り出す `TerminalPair` view を定めるが, これは第二の source judgment ではない。成功 run の `ResolvedContext` / resolved target と, 任意 derivation の terminal pair は, state bisimulation の forward / reverse residual により相互に $\preceq_{\text{ctx}}$ である。closed context では context 成分が空のままなので, 上の target principality へ戻る。

7.8 Damas–Milner 断片への closed 保守性

pattern-free · capability-inert · direct-self 断片の逆方向は、公開の監査済み `SourceTyping` を境界として機械化する。型消去 $[\tau]$ は関数・積を一 sort に構造的に写し、data を component の積へ、基本型と skolem を DM の `Int` へ正規化し、matcher / slot wrapper と capability を忘れる。context 関係 `ContextErases`(Γ, Δ) は、core scheme の各 value-flow instance が DM scheme の instance に消去できることと、free target variable 列の一致を保存する。SchemesClosed から global target variable が空であることを使うと、let の core generalization は DM generalization に写る。

`InFragmentExpr`(e) の下で、`TypingInvariant` の各構成子を構造的に `DM.Typing` へ消去する。coercion 構成子は matcher / slot wrapper の消去で同じ DM 導出に戻り、fragment classifier は constructor, primitive, matcher, pattern-bodied recursion を排除する。公開定理は

$$\begin{aligned} & \text{FrozenSigWF}(\widehat{\Sigma}) \wedge \text{InFragmentExpr}(e) \wedge \text{SourceTyping}(\widehat{\Sigma}, \emptyset, e, \tau) \\ & \implies \text{DM.Typing}(\emptyset, e, [\tau]) \end{aligned}$$

である。左辺から内部 invariant を得る段階は terminal audit による closed state erasure であり、`TypingInvariant` の存在を caller premise にしない。この定理は open context の converse や、消去前後の principal target の構文的一致を主張しない。

7.9 機械化の範囲と非主張

全 expression / pattern / arm / clause の raw family, 対応する Origin family, solver と ledger の局所代数, 全 14 family の state factorization と solved-form 保存, terminal audit, audit を含む `SourceTyping` と実行推論との双方向接続, closed-program state erasure, concrete dynamic safety, 受理同値, decidability, target 一意性, 二 sort instance preorder, target principality, context 相対 principality は Lean 4 に機械化済みである。主定理と一般補題に sorry, admit, project-defined axiom はない。

一般の program termination と、open context で入力 meta をすべて固定する target 一意性は主張しない。また、terminal audit を除いた独立の宣言的型付け、それに対する完全性と安全性、および audit による拒否範囲の正確な特徴付けも現時点では主張しない。既知の matcher-finalization 反例の修正には、matcher 自身が観察した構造と borrowed leaf の由来を終端まで区別する必要があり、borrowed variable の一律 freeze や全 opaque leaf の一律許可は採用していない。pattern-free direct-self Damas–Milner 断片から `TypingInvariant` への埋込みと具体的な受理例に加え、pattern-free syntax と capability-inert type image の実行可能な特徴付け、一 sort substitution の有限 scope 代数, canonical scheme opening の principality, closed audited source typing から DM typing への capability 消去を機械化している。全 `DM.Typing` derivation の executable acceptance は今後の課題である。